Plan: konceptualny ERD dla MobiGo (`reality.md`)

Context

Mamy opis rzeczywistości biznesowej firmy MobiGo (wynajem rowerów elektrycznych i hulajnóg na minuty) w `reality.md`. Trzeba z niego zbudować **konceptualny** diagram ERD wraz z dokumentacją decyzji. Model jest ustalany w tym planie. Kolejny agent **tylko implementuje** ustalenia i **flaguje nieścisłości** — nie podejmuje samodzielnie decyzji modelowych.

Wynik: jeden plik `erd.md` w katalogu głównym repo, diagram w Mermaid `erDiagram`.

Zasady obowiązujące agenta (twarde)

1. **Poziom konceptualny.** Bez kluczy obcych, bez typów SQL, bez tabel pośrednich/technicznych, bez indeksów.
2. **Zero halucynacji.** Każda encja, atrybut i związek musi mieć pokrycie w konkretnym zdaniu `reality.md` (Z1–Z20, niżej). Element bez pokrycia = nie dodawać; wpisać do sekcji „Nieścisłości".
3. **Nie rozstrzygać samemu.** Gdy coś w tym planie jest sprzeczne z `reality.md` albo niejednoznaczne — zrobić wg planu i dopisać wpis do „Nieścisłości" (co, gdzie, dlaczego, proponowane rozwiązanie).
4. **Jawne założenia.** Każde przyjęte założenie z planu trafia do sekcji „Założenia" w `erd.md`.
5. Nazewnictwo: polskie, WIELKIE_LITERY dla encji, snake_case dla atrybutów, bez polskich znaków w identyfikatorach Mermaid.

Krok 1 — Numeracja zdań opisu (do śledzenia pokrycia)

Id	Treść (skrót)
Z1	MobiGo: wynajem rowerów elektrycznych i hulajnóg na minuty
Z2	Nowy użytkownik musi się zarejestrować w aplikacji
Z3	Przy rejestracji podaje imię, nazwisko, e-mail, telefon
Z4	E-mail unikalny, służy jako login
Z5	System nadaje unikalny identyfikator i zapisuje datę rejestracji
Z6	Po zalogowaniu musi dodać metodę płatności, aby opłacać przejazdy
Z7	Może dodać jedną lub więcej kart: numer, data ważności, CVC
Z8	Dane przechowywane bezpiecznie
Z9	W mieście są pojazdy: rowery elektryczne i hulajnogi
Z10	Pojazd ma unikalny numer identyfikacyjny (= kod QR)

Id	Treść (skrót)
Z11	Pojazd: typ, poziom baterii (%), status (dostępny/wypożyczony/w serwisie), ostatnia lokalizacja GPS
Z12	Skan QR rozpoczyna wypożyczenie
Z13	Zapis: który użytkownik, który pojazd, data i godzina rozpoczęcia
Z14	Status pojazdu → „wypożyczony”
Z15	Zakończenie: data i godzina zakończenia, końcowa lokalizacja
Z16	Status pojazdu → „dostępny”
Z17	Po zakończeniu system generuje płatność
Z18	Koszt = czas trwania × cennik dla typu pojazdu (stawka za minutę)
Z19	Płatność: kwota, data transakcji, status (udana/nieudana)
Z20	Płatność powiązana z wypożyczeniem i wybraną metodą płatności

Krok 2 — Encje ✅ USTALONE

Encja	Pokrycie	Uwagi
UZYTKOWNIK	Z2–Z5	
KARTA_PLATNICZA	Z6–Z8	„metoda płatności” = synonim w słowniku; opis zna tylko karty, więc bez nadtypu
POJAZD	Z9–Z11	
TYP_POJAZDU	Z9, Z11, Z18	niesie stawkę za minutę (cennik); bez historii cen (brak pokrycia)
WYPOZYCZENIE	Z12–Z15	
PLATNOSC	Z17–Z20	

Lokalizacja GPS **nie jest** encją — to atrybut złożony (szerokość, długość). Nie dodawać: CENNIK jako osobnej encji, LOKALIZACJA, METODA_PLATNOSCI, STACJA, STREFA, PROMOCJA itp.

Krok 3 — Atrybuty i identyfikatory ✅ USTALONE

W miejscu typu Mermaid wpisujemy **dziedzinę pojęciową**, nie typ SQL. Dozwolone dziedziny: `tekst`, `data`, `dataczas`, `kwota`, `procent`, `gps`, `status`, `identyfikator`, `minuty`. Markery: tylko `PK` (identyfikator) i `UK` (unikalność). **Nigdy `FK`.** Komentarz `"..."` = źródło (Z#) i uwagi.

Encja	Atrybut	Dziedzina	Marker	Pokrycie / uwagi
UZYTKOWNIK	id_uzytkownika	identyfikator	PK	Z5
	imie, nazwisko, telefon	tekst		Z3

Encja	Atrybut	Dziedzina	Marker	Pokrycie / uwagi
	email	tekst	UK	Z3, Z4 (login)
	data_rejestracji	data		Z5
KARTA_PLATNICZA	numer_karty	tekst	PK	Z7; **założenie A1**
	data_waznosci	data		Z7
	kod_cvc	tekst		Z7; **nieścisłość N1** (PCI DSS)
TYP_POJAZDU	nazwa	tekst	PK	Z11 (rower / hulajnoga)
	stawka_za_minute	kwota		Z18
POJAZD	nr_identyfikacyjny	identyfikator	PK	Z10 (= kod QR, jeden atrybut)
	poziom_baterii	procent		Z11
	status	status		Z11 (dostępny / wypożyczony / w serwisie)
	ostatnia_lokalizacja	gps		Z11; atrybut złożony (szerokość, długość)
WYPOZYCZENIE	data_godzina_rozpoczecia	dataczas	PK	Z13; **założenie A2**
	data_godzina_zakonczenia	dataczas		Z15; pusta dla trwającego wypożyczenia
	lokalizacja_koncowa	gps		Z15; atrybut złożony
	czas_trwania	minuty		Z18; **atrybut pochodny** = zakończenie – rozpoczęcie, oznaczyć w komentarzu
PLATNOSC	data_transakcji	dataczas	PK	Z19; **założenie A3**
	kwota	kwota		Z19
	status	status		Z19 (udana / nieudana)

Identyfikatory naturalne (do sekcji „Założenia"):

- **A1** KARTA_PLATNICZA identyfikowana numerem karty (ta sama karta nie występuje u dwóch użytkowników).
- **A2** WYPOZYCZENIE identyfikowane przez POJAZD + data_godzina_rozpoczecia (pojazd nie może mieć dwóch wypożyczeń rozpoczętych w tej samej chwili) → encja słaba, związek z POJAZD

identyfikujący (linia ciągła `--` w Mermaid).
- **A3** PLATNOSC identyfikowana przez WYPOZYCZENIE + data_transakcji → encja słaba, związek z WYPOZYCZENIE **identyfikujący** (`--`).
- Pozostałe związki **nieidentyfikujące** (linia przerywana `..`).

Nie dodawać sztucznych `id_*` poza id_uzytkownika (ten jest w Z5).

Krok 4 — Związki i liczności ✅ USTALONE

Notacja Mermaid (crow's foot): `||` dokładnie 1, `o{` 0..N. `--` identyfikujący, `..` nieidentyfikujący. Etykieta związku = czasownik po polsku, czytany od lewej encji.

#	Mermaid	Znaczenie	Pokrycie
R1	`UZYTKOWNIK		..o{ KARTA_PLATNICZA : posiada`	
R2	`UZYTKOWNIK		..o{ WYPOZYCZENIE : dokonuje`	
R3	`POJAZD		--o{ WYPOZYCZENIE : "jest wypozyczany w"`	
R4	`TYP_POJAZDU		..o{ POJAZD : klasyfikuje`	
R5	`WYPOZYCZENIE		--o{ PLATNOSC : "jest rozliczane przez"`	
R6	`KARTA_PLATNICZA		..o{ PLATNOSC : obciazana`	

Brak związków N:M → nie ma potrzeby encji asocjacyjnych (WYPOZYCZENIE już pełni tę rolę między UZYTKOWNIK i POJAZD).

Krok 5 — Reguły biznesowe i nieścisłości ✅ USTALONE

Reguły (mają pokrycie, nie da się ich pokazać na diagramie):

- **B1** e-mail unikalny, służy jako login (Z4).
- **B2** wypożyczenie wymaga posiadania ≥ 1 karty (Z6).
- **B3** rozpoczęcie → status pojazdu „wypożyczony" (Z14); zakończenie → „dostępny" (Z16).
- **B4** kwota = czas_trwania × stawka_za_minute typu wypożyczonego pojazdu (Z18).
- **B5** płatność powstaje dopiero po zakończeniu wypożyczenia (Z17).
- **B6** karta płatności musi należeć do użytkownika, który dokonał wypożyczenia (Z20) — ograniczenie cyklu UZYTKOWNIK→WYPOZYCZENIE→PLATNOSC→KARTA_PLATNICZA→UZYTKOWNIK.
- **B7** data_godzina_zakonczenia > data_godzina_rozpoczecia.

Nieścisłości wpisane z góry (pytania do klienta; agent dopisuje kolejne od N9):

- **N1** przechowywanie CVC (Z7–Z8) sprzeczne z PCI DSS — w modelu zostaje, bo ma pokrycie.
- **N2** brak reguły ponawiania nieudanej płatności — stąd R5 0..N.
- **N3** sposób zaokrąglania czasu (minuty rozpoczęte czy pełne).
- **N4** czy cennik ma historię („obowiązujący") — model jej nie przewiduje.
- **N5** czy lokalizacja_koncowa aktualizuje ostatnia_lokalizacja pojazdu.
- **N6** czy można wypożyczyć pojazd o statusie innym niż „dostępny".
- **N7** czy użytkownik może mieć kilka wypożyczeń jednocześnie.

- **N8** co z kartą wygasłą/usuniętą, do której są przypisane historyczne płatności.

N5 i N6 celowo **nie** są regułami — brak pokrycia w opisie.

Krok 6 — Napisanie `erd.md` (po polsku) ✅ USTALONE

Jeden plik `erd.md` w katalogu głównym repo, sekcje w tej kolejności:

1. **Cel i poziom modelu** — konceptualny, Mermaid crow's foot; legenda: `||`/`o{`, `--`/`..`, lista dziedzin pojęciowych, brak FK/typów SQL.
2. **Diagram** — blok ```` ```mermaid ```` z `erDiagram`; 6 encji z kroku 2, atrybuty z kroku 3 (format `dziedzina nazwa PK|UK "Z#; uwaga"`), związki R1–R6 dokładnie jak w kroku 4. Atrybut pochodny `czas_trwania` z komentarzem `"pochodny: zakonczenie - rozpoczecie"`.
3. **Słownik encji** — nazwa, definicja jednym zdaniem, pokrycie Z#, synonimy („metoda płatności" → KARTA_PLATNICZA, „cennik" → TYP_POJAZDU.stawka_za_minute, „kod QR" → POJAZD.nr_identyfikacyjny).
4. **Atrybuty** — tabela z kroku 3.
5. **Związki i liczności** — R1–R6 z uzasadnieniem liczności min/max.
6. **Reguły biznesowe** — B1–B7.
7. **Założenia** — A1–A3 oraz konwencja dziedzin pojęciowych.
8. **Nieścisłości** — N1–N8 + nowe (N9…) znalezione podczas implementacji; format: opis, źródło Z#, wpływ na model, proponowane pytanie do klienta.
9. **Macierz pokrycia** — każde Z1–Z20 → lista elementów modelu (E/atrybut/R/B/A/N) albo „brak — uzasadnienie".
10. **Log decyzji** — przyjęte i odrzucone warianty z uzasadnieniem:
    - TYP_POJAZDU zamiast osobnej encji CENNIK albo samej reguły (stawka potrzebuje miejsca w modelu);
    - KARTA_PLATNICZA zamiast METODA_PLATNOSCI / nadtypu (opis zna tylko karty);
    - lokalizacja jako atrybut złożony, nie encja (brak własnej tożsamości);
    - dziedziny pojęciowe zamiast typów SQL (Mermaid wymaga typu, model ma zostać konceptualny);
    - identyfikatory naturalne A1–A3 zamiast sztucznych id (brak pokrycia dla id);
    - kod_cvc zostaje + N1 (ma pokrycie, konflikt z praktyką zgłoszony);
    - czas_trwania jako atrybut pochodny;
    - R1 0..N + B2 (konto istnieje przed dodaniem karty);
    - R5 0..N + N2 (status „nieudana" sugeruje ponowienia);
    - N5/N6 jako nieścisłości, nie reguły (brak pokrycia).

Rola agenta: implementuje dokładnie powyższe. Jeśli coś w planie jest sprzeczne z `reality.md`, z samym sobą albo z Mermaid — implementuje wg planu i dopisuje N9+; **nie zmienia modelu na własną rękę**.

Krok 7 — Weryfikacja ✅ USTALONE

1. **Render:** wyciąć blok Mermaid do pliku w scratchpadzie i uruchomić `npx -y @mermaid-js/mermaid-cli -i <plik>.mmd -o <plik>.svg` — musi przejść bez błędu składni. Gdy render niedostępny (np. brak Chromium) — zapisać to w raporcie; nie obchodzić zmianą modelu.
2. **Lista kontrolna (grep po `erd.md`):**

- brak `FK`, brak typów SQL (`int`, `varchar`, `decimal`, `timestamp`…);
- dokładnie 6 encji i 6 związków; tylko dziedziny z listy kroku 3;
- każdy atrybut i związek ma Z# w komentarzu/tabeli;
- macierz pokrycia zawiera wszystkie Z1–Z20;
- każde A# i N# z planu występuje w `erd.md`.

3. **Raport końcowy agenta:** co zrobiono, wynik renderu, lista nowych nieścisłości (N9+), jeśli są.